Mastering Kubernetes RBAC

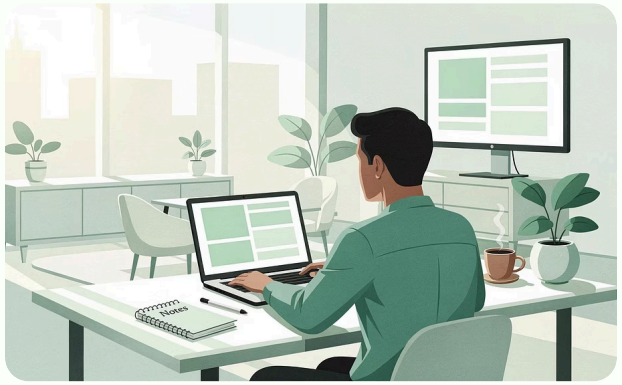
User Accounts, Service Accounts, and Permissions – a comprehensive guide to securing your Kubernetes cluster through Role-Based Access Control.

KUBERNETES SECURITY

RBAC

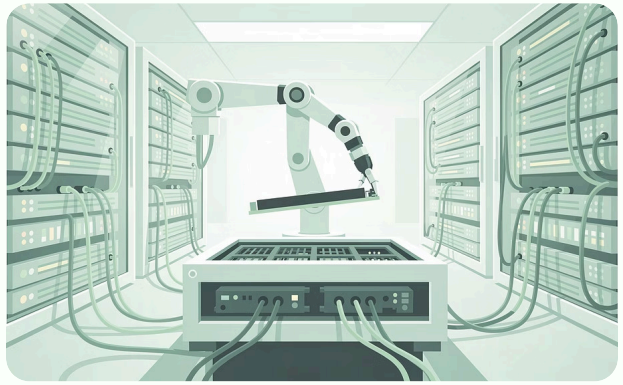
The Foundation: Who Can Do What?

Before assigning permissions, Kubernetes needs to know **who** is making a request. There are two fundamental identity types in every cluster:



User Accounts

Represent **human users** interacting with the cluster – such as developers, operators, or administrators. These are typically managed externally (e.g., via certificates or identity providers) and are not stored as Kubernetes objects.



Service Accounts

Represent **applications or processes** running inside the cluster that need to communicate with the Kubernetes API. Unlike user accounts, service accounts are first-class Kubernetes objects, created and managed within a namespace.

Defining Permissions: The Power of Roles

Roles are the building blocks of RBAC. They define **what actions** (verbs) can be performed on **which resources**. There are two kinds, distinguished by their scope:

Role

Scoped to a **single namespace**. Defines permissions for resources like pods, deployments, or services within that namespace only. Ideal for team-level or application-level access control.

- Verbs: get, list, create, delete
- Resources: pods, deployments, configmaps

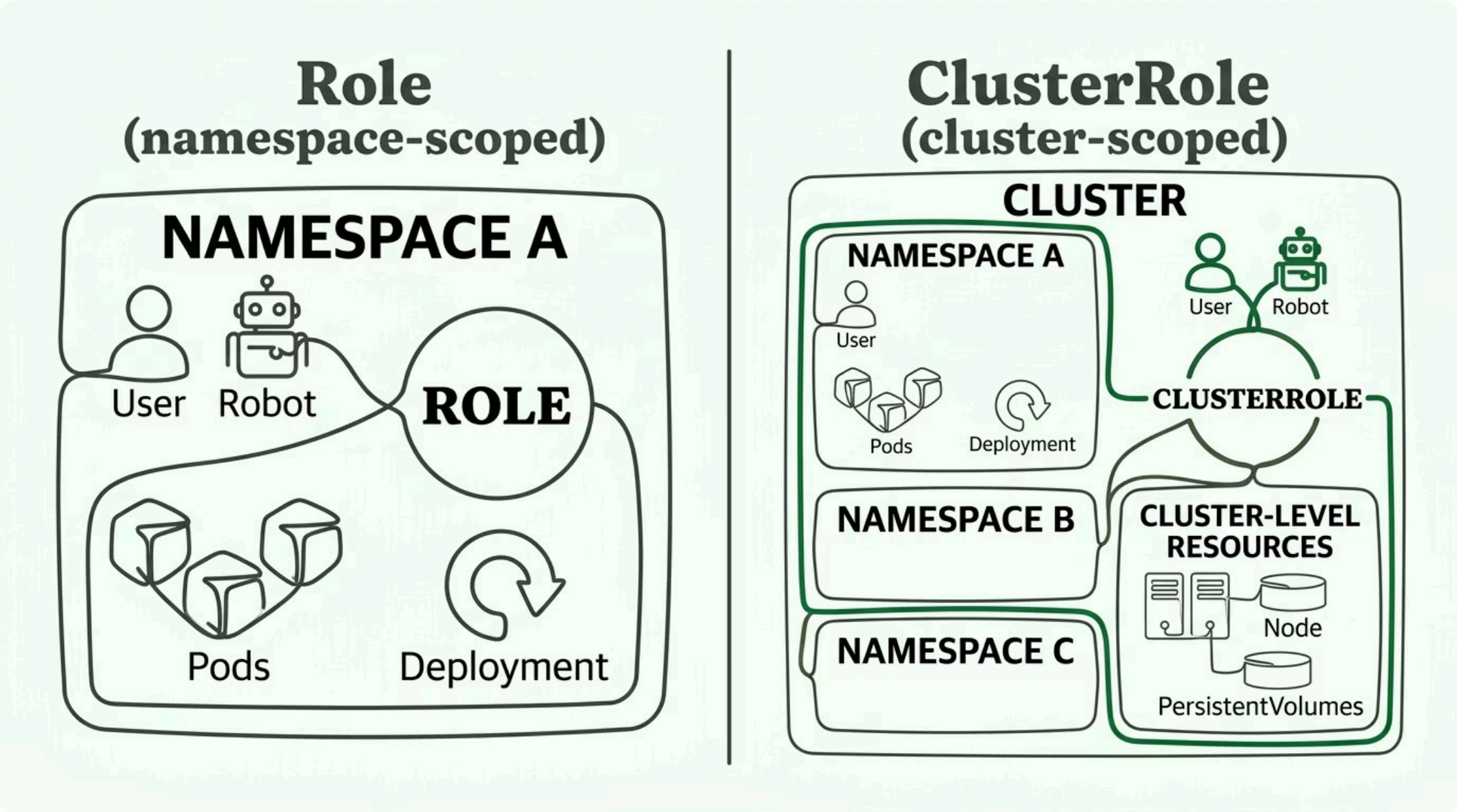
ClusterRole

Applies **across all namespaces** or to cluster-scoped resources such as nodes, persistentvolumes, and namespaces themselves. Used when permissions must span the entire cluster.

- Access to cluster-scoped resources
- Can be reused across multiple namespaces

Roles in Context: Namespace vs. Cluster Scope

This diagram illustrates how **Roles** are confined to a single namespace, while **ClusterRoles** span the entire cluster – including all namespaces and cluster-level resources.



A **ClusterRole** can also be used within a single namespace when bound via a **RoleBinding** – giving you flexibility to define reusable permission sets at the cluster level and apply them locally.

Binding Roles to Subjects: RoleBindings

A **RoleBinding** connects a Role (or ClusterRole) to one or more **subjects** – users, groups, or service accounts – but only within a **single namespace**. This is the most common binding for day-to-day access control.

→ Subject

A user, group, or service account that will receive the permissions.

→ Role Reference

Points to the Role (or ClusterRole) containing the permission rules.

→ Namespace Scope

Permissions are enforced *only* within the specified namespace.

Example Use Case

Granting a developer **read-only access** to pods exclusively in the `development` namespace – without any visibility into `production` or other namespaces.

Subjects: `dev-user` **Role:** `pod-reader` **Namespace:** `development`

Binding Roles to Subjects: ClusterRoleBindings

A **ClusterRoleBinding** grants the permissions defined in a ClusterRole to subjects **across the entire cluster** – unrestricted by namespace boundaries. Use this sparingly and only when truly cluster-wide access is required.

Cluster-Wide Scope

Permissions apply to all namespaces and all cluster-scoped resources simultaneously.

Admin Use Case

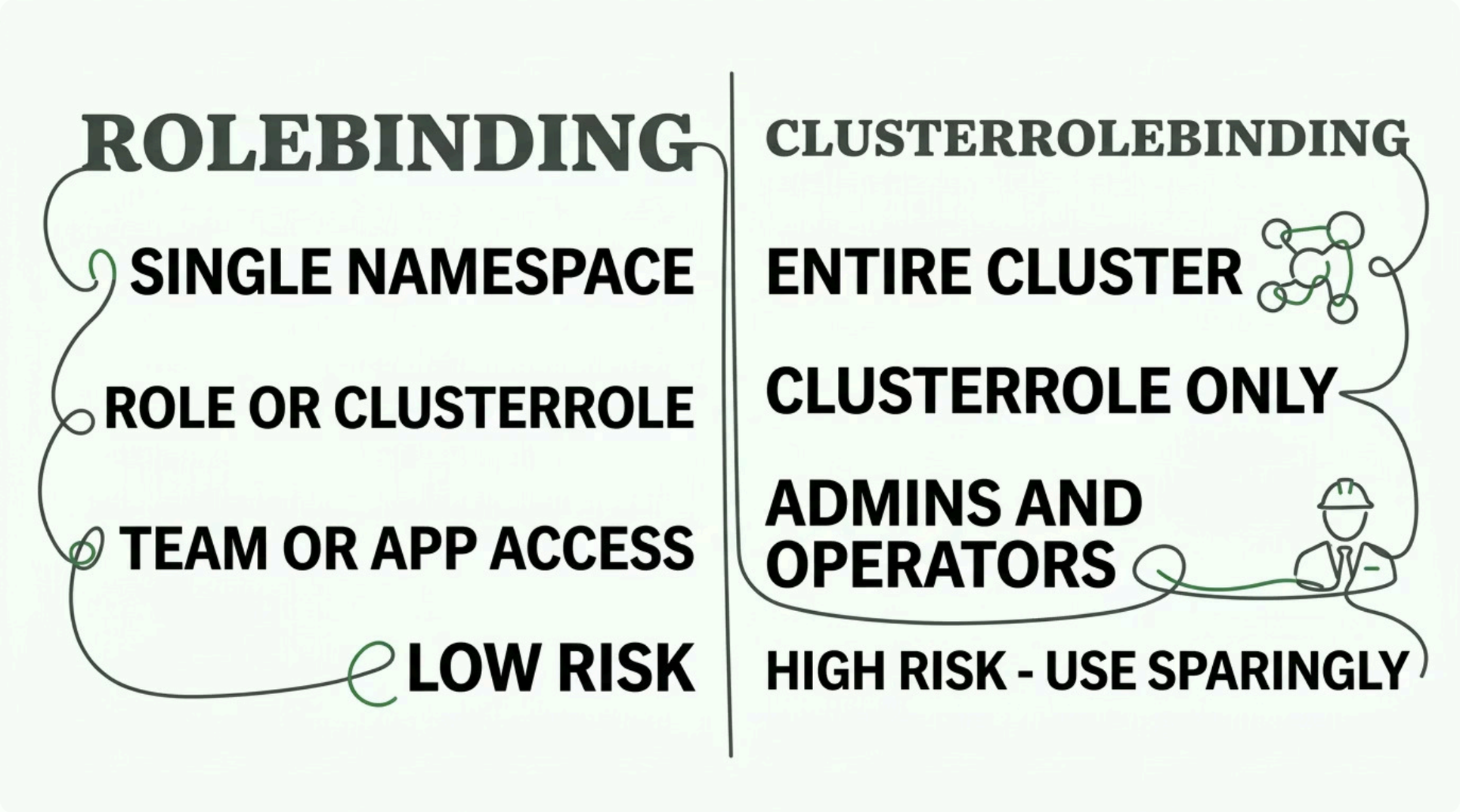
Granting a cluster administrator **full access** to all resources in all namespaces – the most powerful binding type.

Use With Caution

ClusterRoleBindings are powerful. Over-provisioning cluster-wide access is a common security risk – always prefer namespace-scoped bindings where possible.

Key Distinction: Scope is Everything

Choosing between a RoleBinding and a ClusterRoleBinding is fundamentally a question of **scope**. Understanding this distinction is critical to implementing least-privilege security correctly.



Pro Tip: A RoleBinding can reference a ClusterRole – this lets you define a reusable permission template at the cluster level and apply it in a namespace-specific context, combining flexibility with scope control.

Summary: Secure Your Cluster with RBAC

RBAC is the cornerstone of Kubernetes security. Mastering these four building blocks gives you precise, auditable control over every action in your cluster.

1

Identify Your Subjects

Use **User Accounts** for humans and **Service Accounts** for workloads needing API access.

2

Define Permissions

Use **Roles** for namespace-scoped rules and **ClusterRoles** for cluster-wide or reusable permissions.

3

Bind Appropriately

Use **RoleBindings** for namespace access and **ClusterRoleBindings** only when truly cluster-wide scope is needed.

4

Least Privilege

Always grant the **minimum permissions** necessary. Regularly audit bindings and remove unused access.

🔑 **Key Takeaway:** The principle of least privilege is not just a best practice – in Kubernetes, it is your primary defense against lateral movement and accidental damage in a shared cluster environment.